



Swin Transformer: Hierarchical Vision Transformer using Shifted Windows

1. Introduction

논문이 다루는 분야

- **분야:** 컴퓨터 비전을 위한 범용 백본(general-purpose backbone) 설계.
 - 분류(classification), 객체 탐지(object detection), 의미론적 분할(semantic segmentation) 등 다양한 비전 태스크에 두루 쓰일 수 있는 백본
- **배경:** NLP에서 Transformer가 표준이 된 반면, 비전에서는 여전히 CNN이 주류였음.
 - ViT가 이미지 분류에서 Transformer의 가능성을 보였으나, 범용 백본으로 쓰기에는 한계가 있었음.
- **?** 언어 도메인에서의 Transformer 성공을 비전 도메인으로 어떻게 옮길 것인가
→ 두 도메인의 본질적 차이를 분석, 이를 해소하는 구조 제안

해당 task에서 기존 연구 한계점

NLP vs. Vision 두 모달리티에서 오는 한계점

1. 스케일(scale)의 다양성

- 언어의 단어 토큰은 기본 처리 단위로서 크기가 고정적임.
- 반면 시각적 객체는 크기 변화가 매우 큼.
(예: 같은 장면 속 작은 보행자와 큰 자동차)
- 기존 Transformer 기반 모델(ViT 등)은 모든 토큰이 **고정된 단일 스케일**이라, 이러한 다중 스케일 특성을 다루기 부적합했음.

2. 고해상도(high resolution)로 인한 연산량 폭증

- 이미지의 픽셀 수 >>> 문장의 단어 수
- Self-attention의 연산 복잡도는 토큰 수에 대해 **제곱(quadratic)**으로 증가함
- 따라서 의미론적 분할처럼 픽셀 단위 밀집 예측(dense prediction)이 필요한 고해상도 태스크에서는 전역 self-attention이 사실상 계산 불가능함.

3. ViT의 구조적 제약

- ViT는 **단일 저해상도** 특징 맵만 생성해 밀집 예측 태스크에 직접 활용 힘들.
- 또한 입력 이미지 크기에 대해 복잡도가 제곱(quadratic)으로 증가함.

논문의 contributions

- **계층적 표현(hierarchical representation)**
 - 작은 패치에서 시작해 깊은 층으로 갈수록 인접 패치를 병합하여, CNN처럼 다중 해상도 특징 맵을 생성
 - FPN, U-Net 등 기존 밀집 예측 프레임워크에 그대로 결합 가능
- **Shifted Window 기반 self-attention**
 - 겹치지 않는 지역 윈도우 내부로 self-attention을 한정 → 이미지 크기에 대해 **선형(linear)** 복잡도 달성 + 윈도우를 층마다 이동시켜 윈도우 간 연결성을 확보
- **광범위한 SOTA 달성**
 - 이미지 분류(ImageNet-1K 87.3% top-1)
 - 객체 탐지(COCO test-dev 58.7 box AP / 51.1 mask AP)
 - 의미론적 분할(ADE20K 53.5 mIoU)
 - 다양한 태스크에서 모두 당시 최고 성능을 큰 폭으로 경신
 - 제한한 계층적 설계와 shifted window 기법이 all-MLP 구조(MLP-Mixer 계열)에도 효과적임을 입증함.

2. Related Work

- **CNN 및 변형**
 - AlexNet 이후 VGG, GoogLeNet, ResNet, DenseNet, EfficientNet 등으로 발전해 온 비전의 표준 백본 계열임.

→ 이를 대체할 Transformer 기반 통합 모델링의 가능성을 강조

- **Self-attention 기반 백본**

- ResNet의 일부/전체 공간 합성곱 층을 self-attention으로 대체하려는 시도들이 있었음.
- 각 픽셀의 지역 윈도우 내에서 attention을 계산했으나, **sliding window** 방식의 비효율적 메모리 접근 때문에 실제 지연 시간(latency)이 CNN보다 훨씬 컸음.
- sliding → **층 간 윈도우 이동(shift)** : 일반 하드웨어에서 효율적 구현을 달성.

- **CNN을 보완하는 self-attention/Transformer**

- 백본이나 헤드 네트워크에 self-attention을 더해 원거리 의존성을 보완
- DETR처럼 검출/분할에 encoder-decoder Transformer를 적용
- Swin Transformer : 기초 시각 특징 추출을 Transformer로 수행

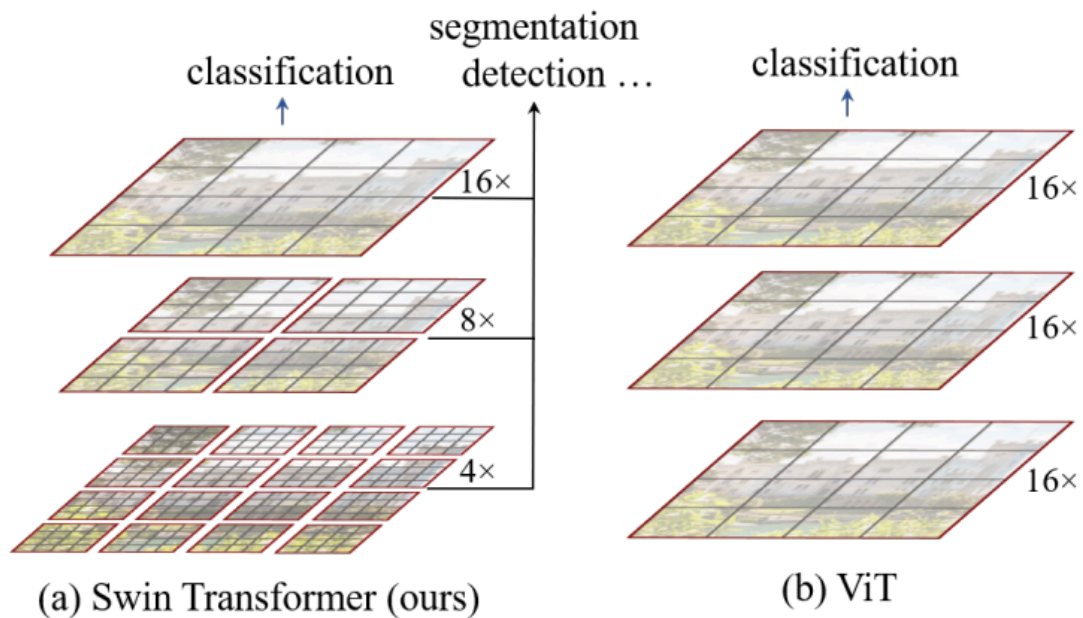
- **Transformer 기반 비전 백본**

- 가장 밀접한 선행 연구는 ViT계열 (DeiT 등)
 - ViT는 대규모 데이터(JFT-300M), DeiT는 ImageNet-1K 학습에 요구
 - 둘 다 저해상도 단일 특징 맵과 제곱 복잡도라는 한계가 있었음.
 - 동시기 연구(PVT 등)도 다중 해상도 맵을 시도
 - 복잡도가 여전히 제곱
 - Swin은 선형적, 지역적으로 동작함

3. 제안 방법론

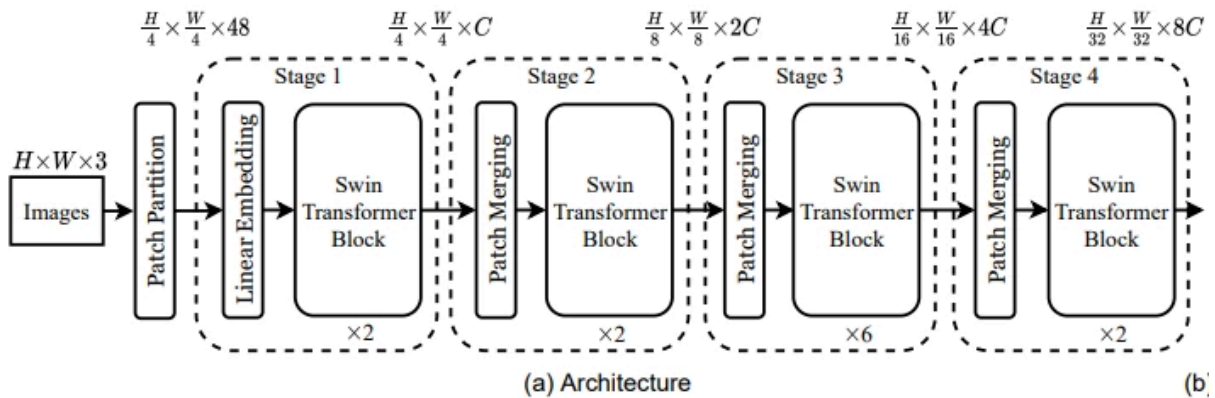
Main Idea

✓ Figure 1 — 계층적 특징 맵 (Swin vs. ViT)



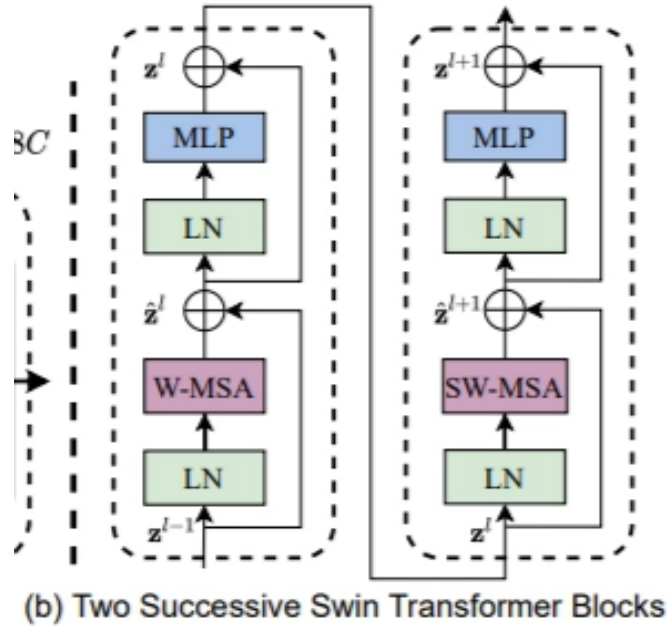
- (a) Swin Transformer
 - 작은 패치(회색)에서 시작 → 깊은 층에서 패치를 병합하며 계층적 특징 맵 생성
 - self-attention을 각 지역 윈도우(빨간색) 안에서만 계산 → 이미지 크기에 대해 선형 복잡도
 - 4x, 8x, 16x 등 여러 해상도 생성 → 분류와 밀집 예측 모두에 활용 가능
- (b) 기존 ViT
 - 단일 저해상도 특징 맵만 생성
 - self-attention을 전역으로 계산 → 이미지 크기에 대해 제곱 복잡도

✓ Figure 3 — 전체 아키텍처 (Swin-T 기준)



- Patch Partition
 - 입력 RGB 이미지를 겹치지 않는 4×4 패치로 분할, 각 패치 = 토큰
 - 특징 = 원시 RGB 값의 연결 → 차원 4×4×3 = 48
- Stage 1
 - Linear Embedding으로 48차원 → 임의 차원 C 로 투영 후 Swin 블록 적용
 - 토큰 수 $\frac{H}{4} \times \frac{W}{4}$ 유지
- Stage 2~4
 - 각 단계 시작에 Patch Merging 층: 2×2 인접 패치 연결(4C) → 선형 층으로 2C로 축소
 - 결과: 토큰 수 1/4 (해상도 2× 다운샘플링), 채널 2배
 - 해상도 $\frac{H}{8}, \frac{H}{16}, \frac{H}{32}$ (각 변 기준)로 단계적 축소
- 다중 해상도 표현 = VGG·ResNet과 동일한 특징 맵 해상도
 - 기존 비전 프레임워크 백본을 그대로 대체 가능

✓ Figure 3(b) — 두 개의 연속된 Swin Transformer 블록



- Swin 블록 = 표준 MSA를 shifted window 기반 모듈로 교체 (나머지 구성은 동일)
- 블록 구성: LN → (S)W-MSA → 잔차 연결 → LN → 2-layer MLP(GELU) → 잔차 연결
- 윈도우 방식은 항상 쌍으로 적용
 - 첫 블록 : W-MSA (일반 윈도우)
 - 다음 블록 : SW-MSA (이동된 윈도우)

- 수식 (3):

$$\hat{z}^l = \text{W-MSA}(\text{LN}(z^{l-1})) + z^{l-1}$$

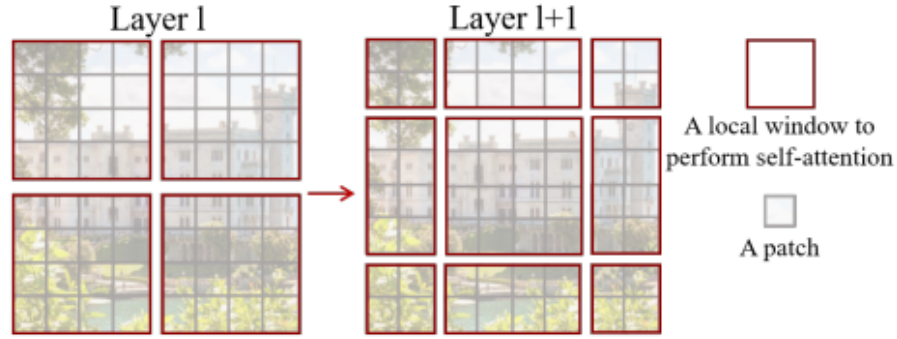
$$z^l = \text{MLP}(\text{LN}(\hat{z}^l)) + \hat{z}^l$$

$$\hat{z}^{l+1} = \text{SW-MSA}(\text{LN}(z^l)) + z^l$$

$$z^{l+1} = \text{MLP}(\text{LN}(\hat{z}^{l+1})) + \hat{z}^{l+1}$$

- $\hat{z}^l, z^l$: 각각 블록 l 의 (S)W-MSA 모듈과 MLP 모듈의 출력 특징

✓ Figure 2 — Shifted Window 접근법



- 층 l (좌) : 일반 윈도우 분할
 - 8×8 특징 맵을 좌상단부터 4×4 크기의 2×2 윈도우로 균등 분할
 - 각 윈도우 내부에서 self-attention 계산
- 층 $l + 1$ (우) : 이동된 윈도우 분할
 - 분할을 ($\lfloor \frac{M}{2} \rfloor, \lfloor \frac{M}{2} \rfloor$) 픽셀만큼 이동(shift)
 - 새 윈도우가 이전 층 윈도우 경계를 가로지름 → 윈도우 간 연결(cross-window connection) 생성 → 모델링 능력 향상

✓ 비-중첩 윈도우 내 self-attention의 복잡도 (수식 1, 2)

- $h \times w$ 패치 이미지, 각 윈도우가 $M \times M$ 패치 포함 시:

$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C$$

- 전역 MSA : 패치 수 hw 에 대해 제곱

$$\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC$$

- 윈도우 기반 W-MSA : M 고정($M = 7$ 기본값)이면 선형 → 고해상도 확장의 핵심 근거

✓ Figure 4 — 이동 구성을 위한 효율적 배치 연산 (cyclic shift)

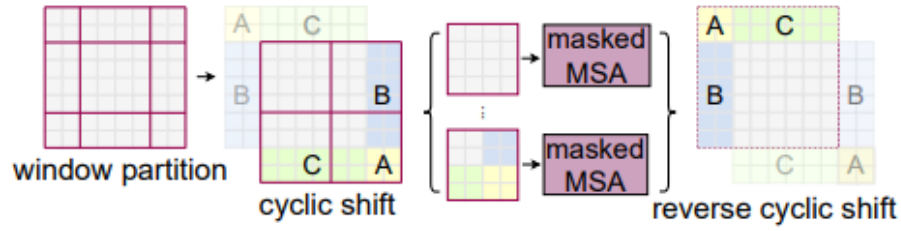


Figure 4. Illustration of an efficient batch computation approach for self-attention in shifted window partitioning.

- 문제
 - 윈도우 이동 시 윈도우 개수가 $\lceil \frac{h}{M} \rceil \times \lceil \frac{w}{M} \rceil \rightarrow (\lceil \frac{h}{M} \rceil + 1) \times (\lceil \frac{w}{M} \rceil + 1)$ 로 증가
 - 일부 윈도우는 $M \times M$ 보다 작아짐
 - 단순 패딩 : $2 \times 2 \rightarrow 3 \times 3$ 처럼 연산 2.25배 증가 $\rightarrow$ 비효율적
- 해결
 - 특징 맵을 좌상단 방향으로 cyclic shift
 - 인접하지 않은 하위 윈도우가 한 배치 윈도우에 섞임 $\rightarrow$ masking으로 각 하위 윈도우 내부로만 self-attention 제한
 - 이후 reverse cyclic shift로 원위치
- 효과 : 배치 윈도우 수가 일반 분할과 동일하게 유지 $\rightarrow$ 효율적, 낮은 지연 시간(Table 5)

✓ Relative Position Bias (수식 4)

- self-attention 유사도 계산 시 각 head에 상대 위치 편향 B 를 더함:

$$\text{Attention}(Q, K, V) = \text{SoftMax}(QK^T / \sqrt{d} + B)V$$

- $Q, K, V \in \mathbb{R}^{M^2 \times d}$: query/key/value 행렬, d : query/key 차원, M^2 : 윈도우 내 패치 수
- 각 축 상대 위치 범위 $[-M + 1, M - 1]$ $\rightarrow$ 작은 편향 행렬 $\hat{B} \in \mathbb{R}^{(2M-1) \times (2M-1)}$ 을 파라미터화해 B 값을 가져옴
- 절대 위치 임베딩보다 우수 $\rightarrow$ 절대 위치 임베딩 추가 시 오히려 성능 소폭 하락(Table 4)

Contribution

✓ Table 7 — 아키텍처 변형 (입력 224×224 기준)

	downsp. rate (output size)	Swin-T	Swin-S	Swin-B	Swin-L
stage 1	4× (56×56)	concat 4×4, 96-d, LN win. sz. 7×7, dim 96, head 3 × 2	concat 4×4, 96-d, LN win. sz. 7×7, dim 96, head 3 × 2	concat 4×4, 128-d, LN win. sz. 7×7, dim 128, head 4 × 2	concat 4×4, 192-d, LN win. sz. 7×7, dim 192, head 6 × 2
stage 2	8× (28×28)	concat 2×2, 192-d, LN win. sz. 7×7, dim 192, head 6 × 2	concat 2×2, 192-d, LN win. sz. 7×7, dim 192, head 6 × 2	concat 2×2, 256-d, LN win. sz. 7×7, dim 256, head 8 × 2	concat 2×2, 384-d, LN win. sz. 7×7, dim 384, head 12 × 2
stage 3	16× (14×14)	concat 2×2, 384-d, LN win. sz. 7×7, dim 384, head 12 × 6	concat 2×2, 384-d, LN win. sz. 7×7, dim 384, head 12 × 18	concat 2×2, 512-d, LN win. sz. 7×7, dim 512, head 16 × 18	concat 2×2, 768-d, LN win. sz. 7×7, dim 768, head 24 × 18
stage 4	32× (7×7)	concat 2×2, 768-d, LN win. sz. 7×7, dim 768, head 24 × 2	concat 2×2, 768-d, LN win. sz. 7×7, dim 768, head 24 × 2	concat 2×2, 1024-d, LN win. sz. 7×7, dim 1024, head 32 × 2	concat 2×2, 1536-d, LN win. sz. 7×7, dim 1536, head 48 × 2

Table 7. Detailed architecture specifications.

- 공통 설정
 - 윈도우 크기 $M = 7$
 - head당 query 차원 $d = 32$
 - MLP 확장 비율 $\alpha = 4$
- 모델별 (C = 첫 단계 은닉층 채널 수, 블록 수 = 4개 단계)
 - Swin-T : $C = 96, \{2, 2, 6, 2\} \rightarrow$ ResNet-50(DeiT-S)급 (약 0.25×)
 - Swin-S : $C = 96, \{2, 2, 18, 2\} \rightarrow$ ResNet-101급 (약 0.5×)
 - Swin-B : $C = 128, \{2, 2, 18, 2\} \rightarrow$ ViT-B/DeiT-B급 (약 1×)
 - Swin-L : $C = 192, \{2, 2, 18, 2\} \rightarrow$ Swin-B의 약 2×

4. 실험 및 결과

Dataset

- ImageNet-1K (이미지 분류)
 - 1,000개 클래스, 학습 128만 / 검증 5만 장
 - single-crop top-1 정확도 보고
- ImageNet-22K (사전학습용)
 - 1,420만 장, 2.2만 클래스
- COCO 2017 (객체 탐지 / 인스턴스 분할)

- 학습 11.8만 / 검증 5천 / test-dev 2만 장
- ADE20K (의미론적 분할)
 - 150개 의미 범주, 총 2.5만 장 (학습 2만 / 검증 2천 / 테스트 3천)

Baseline

- 분류
 - Transformer 계열 : DeiT, ViT
 - ConvNet 계열 : RegNet, EfficientNet
- 탐지
 - 백본 비교 : ResNe(X)t, DeiT
 - 프레임워크 : Cascade Mask R-CNN, ATSS, RepPoints v2, Sparse R-CNN
 - 시스템 레벨 : 개선된 HTC++
- 분할
 - 백본 비교 : ResNet-101, ResNeSt, DeiT, SETR
 - 프레임워크 : UperNet

결과

✓ ImageNet-1K 분류

- (a) 일반 ImageNet-1K 학습
 - Swin-T 81.3% → DeiT-S(79.8%) 대비 +1.5%
 - Swin-B : $224^2/384^2$ 입력에서 DeiT-B 대비 +1.5%/+1.4%
 - RegNet, EfficientNet 대비 약간 더 나은 속도-정확도 균형
- (b) ImageNet-22K 사전학습
 - Swin-B : 약 1.8~1.9% 향상 → 86.4% (ViT 대비 +2.4%, 더 적은 FLOPs)
 - Swin-L : 87.3% top-1

✓ COCO 객체 탐지/인스턴스 분할

- (a) 4개 프레임워크 모두에서 Swin-T가 ResNet-50 대비 일관되게 +3.4~4.2 box AP

- (b) Cascade Mask R-CNN
 - Swin vs. ResNeXt101-64×4d : +3.6 box AP / +3.3 mask AP
 - Swin-T vs. DeiT-S : +2.5 box AP / +2.3 mask AP, 동시에 추론 속도도 빠름(15.3 vs. 10.4 FPS)
- (c) 시스템 레벨 최고 모델 : COCO test-dev 58.7 box AP / 51.1 mask AP
 - 이전 SOTA 대비 +2.7 box AP(Copy-paste), +2.6 mask AP(DetectoRS)

✓ ADE20K 의미론적 분할

- Swin-S : 유사 연산량 DeiT-S 대비 +5.3 mIoU, ResNet-101 대비 +4.4 mIoU
- Swin-L (ImageNet-22K 사전학습) : val 53.5 mIoU → 이전 최고(SETR, 50.3) 대비 +3.2 mIoU

✓ Ablation: Shifted Window & 위치 임베딩

- Shifted window 적용 (vs. 단일 윈도우 분할)
 - ImageNet +1.1% top-1
 - COCO +2.8 box AP / +2.2 mask AP
 - ADE20K +2.8 mIoU
- Relative position bias
 - 무위치 인코딩 및 절대 위치 임베딩보다 우수
 - 절대 위치 임베딩 : 분류엔 +0.4% 도움, 탐지/분할에는 오히려 해로움

✓ Self-attention 구현 방식별 속도/정확도

- Cyclic shift vs. 단순 패딩 : 하드웨어 효율적, Swin-T/S/B에서 각각 13%/18%/18% 속도 향상
- Shifted window vs. sliding window : 훨씬 빠르면서(Table 5) 정확도는 동등(Table 6)
- vs. Performer : 약간 더 빠르면서 ImageNet +2.3% top-1 우위

5. 결론 (배운점)

연구의 의의

- 계층적 특징 표현 + 입력 크기에 대한 선형 복잡도를 동시에 달성한 새로운 비전 Transformer
 - 분류·탐지·분할에서 모두 당시 SOTA를 큰 폭으로 경신
- 핵심 요소 shifted window 기반 self-attention
 - 비전 문제에서 효과적 + 효율적임을 입증
 - NLP에도 적용 가능성 시사
- 비전과 언어의 통합 모델링 가능성을 한층 앞당김
(이후 Swin 계열이 다수 비전 태스크의 표준 백본으로 자리잡음)

한계점 및 배운점

- 한계점
 - ResNe(X)t는 고도로 최적화된 cuDNN 함수로 구현 vs. Swin은 최적화되지 않은 기본 PyTorch 함수로 구현
 - 이론적 FLOPs 우위가 실제 latency로 온전히 이어지지 않는 (커널 최적화는 논문 범위 밖)
- 배운점
 1. 문제를 모달리티 차이로 환원하는 사고 : "스케일 다양성"과 "고해상도"를 명확히 짚고, 각각을 계층적 구조·지역 윈도우로 대응시킴
 2. 엔지니어링 디테일이 곧 성능 : cyclic shift + masking은 이론적 우아함보다 실제 하드웨어 효율을 노린 결정 → 연구와 실용성의 접점
 3. Ablation(Table 4) : 절대 위치 임베딩이 태스크별로 득실이 갈림 → 분류에서 좋은 것이 밀집 예측에서도 좋으리라는 가정을 경계

Generative Adversarial Nets (GAN)

1. Introduction

논문이 다루는 분야

- 분야 : 심층 생성 모델(deep generative model)

- 자연 이미지, 음성 파형, 자연어 말뭉치처럼 AI가 다루는 데이터의 확률 분포를 표현하는 풍부하고 계층적인 모델 학습이 목표
- 배경
 - 당시 딥러닝의 두드러진 성공은 고차원 입력을 클래스 레이블로 매핑하는 판별 모델 (discriminative model)에 집중
 - 역전파(backpropagation), 드롭아웃(dropout), 구간별 선형 유닛(piecewise linear unit)에 크게 기반
- **?** 판별 모델이 누린 성공을, 더디게 발전해 온 생성 모델로 어떻게 가져올 것인가
→ 이 간극을 우회하는 새로운 생성 모델 추정 절차 제안

해당 task에서 기존 연구 한계점

생성 모델이 판별 모델만큼 파급력을 갖지 못한 두 가지 근본 원인

1. 다루기 어려운 확률 계산(intractable computation)
 - 최대우도 추정(MLE) 및 관련 전략에서 등장하는 다루기 힘든 확률 계산을 근사해야 함
 - 예: RBM, DBM 같은 무향 그래프 모델 → 정규화 상수(분할 함수)와 그 기울기가 계산 불가능 → MCMC 같은 근사법에 의존
2. 생성 맥락에서 piecewise linear unit의 이점을 살리기 어려움
 - 판별 모델에서는 잘 작동하는 구간별 선형 유닛의 잘 정의된 기울기 → 생성 맥락에서는 활용 어려움

논문의 contributions

- **Adversarial nets 프레임워크 제안**
 - 생성 모델 G 와 판별 모델 D 를 동시에 학습시키는 적대적(adversarial) 과정
 - G 는 데이터 분포를 포착하고, D 는 샘플이 학습 데이터에서 왔는지 G 에서 왔는지를 판별.
- **직관적 비유 (위조지폐 vs. 경찰)**
 - 생성 모델은 감지되지 않고 가짜 화폐를 만들려는 위조범
 - 판별 모델은 이를 검출하려는 경찰
 - 경쟁을 통해 양쪽이 발전해 위조품이 진품과 구별되지 않을 때까지 개선

- **Minimax 2인 게임**

- 이 프레임워크는 minimax 2인 게임에 대응됨.
- G 와 D 가 임의 함수 공간에서 충분한 용량을 가질 때 유일한 해, 즉 G 가 데이터 분포를 복원하고 D 가 모든 지점에서 $1/2$ 인 해가 존재함을 제시.

- **adversarial nets 특수 사례**

- G 와 D 를 다층 퍼셉트론(MLP)으로 정의하는 특수 사례에서는 전체 시스템을 **역전파와 드롭아웃만으로** 학습 가능 + 샘플 생성은 순전파만으로 가능
- 학습이나 생성 과정에 **마르코프 체인(Markov chain)**이나 근사 추론 네트워크가 전혀 필요 **✗**